织界引擎架构与共识

一.API定义

1️⃣ Pipeline 管理类（核心入口）

✔️创建 Pipeline

`POST /pipelines`

作用：

- 创建一个 workflow 实例

✔️ 启动 Pipeline

`POST /pipelines/{id}/run`

作用：

- 触发 Engine 执行

✔️ 查询状态

`GET /pipelines/{id}`

返回：

```
代码块
1  {
2    "status": "running",
3    "current_stage": "code",
4    "progress": 60%
5  }
```

✔️ 暂停 / 恢复（加分）

`POST /pipelines/{id}/pause`

`POST /pipelines/{id}/resume`

🧑‍⚖️ 3️⃣ Human-in-the-loop（非常关键）

👉 这是比赛重点

✔️ Approve

`POST /checkpoints/{id}/approve`

✔️ Reject

`POST /checkpoints/{id}/reject`

`GET  /context  → debug`

二、状态转换

一、我们只用最简单流程

需求分析 → 方案设计（需要审批）→ 写代码 → 完成

二、定义最少状态

Pipeline 状态

代码块

```
CREATED
RUNNING
WAITING_APPROVAL
FINISHED
```

Stage 状态

代码块

```
PENDING
RUNNING
DONE
WAITING_APPROVAL
```

事件（Event）

这些其实就是你的 API👇

```
create_pipeline      (POST /pipelines)
run_pipeline         (POST /run)
stage_done           (内部事件)
need_approval        (内部事件)
approve              (POST /approve)
reject               (POST /reject)
```

三、核心状态流转（最重要）

🚀 1. 创建 Pipeline

`POST /pipelines`

👉 状态：

`Pipeline: CREATED`

`Stage1(需求分析): PENDING`

🚀 2. 启动流程

`POST /pipelines/{id}/run`

👉 状态变化：

`Pipeline: RUNNING`

`Stage1: RUNNING → DONE`

🚀 3. 进入方案设计

`Stage2: RUNNING → WAITING_APPROVAL`

`Pipeline: WAITING_APPROVAL`

👉 系统停住，等人操作

🚀 4. 用户审批

✔️ 同意

`POST /checkpoints/{id}/approve`

👉 状态变化：

`Stage2: DONE`

`Pipeline: RUNNING`

✔️ 拒绝

`POST /checkpoints/{id}/reject`

👉 状态变化（简单版）：

`Stage2: PENDING`

`Pipeline: RUNNING`

（重新做方案设计）

🚀 5. 继续执行

`Stage3(写代码): RUNNING → DONE`

🚀 6. 完成

`Pipeline: FINISHED`

四、用一张“超简单状态图”总结

代码块

```
POST /pipelines
   ↓
CREATED
   ↓ POST /run
RUNNING
   ↓
[需求分析 DONE]
   ↓
[方案设计 WAITING_APPROVAL]
   ↓
   ├── approve → RUNNING → 继续
   └── reject  → 回到 PENDING
   ↓
[写代码 DONE]
   ↓
FINISHED
```

五、最小伪代码（核心逻辑）

Engine（自动跑）

代码块

```
def run_pipeline(pipeline):
    for stage in pipeline.stages:
        if stage.status == "PENDING":
```

```
            stage.status = "RUNNING"

            if stage.name == "design":
                stage.status = "WAITING_APPROVAL"
                pipeline.status = "WAITING_APPROVAL"
                return  # 停住等人

            stage.status = "DONE"
```

API：approve

代码块

```
def approve(checkpoint_id):
    stage = find_stage_by_checkpoint(checkpoint_id)

    stage.status = "DONE"
    pipeline.status = "RUNNING"

    run_pipeline(pipeline)  # 继续执行
```

API：reject

代码块

```
def reject(checkpoint_id):
    stage = find_stage_by_checkpoint(checkpoint_id)

    stage.status = "PENDING"
    pipeline.status = "RUNNING"

    run_pipeline(pipeline)  # 重新执行
```

六、你现在可以这样理解整个系统

API：改变状态（开始 / approve / reject）

Engine：看到状态变化 → 自动往下跑

Stage：每一步具体做什么

三、stage中的数据结构设计（`context`）

代码块

```
context = {
    # ======================
    # 1️⃣ 原始需求
    # ======================
    "requirement_raw": "",
    "codebase": {
        "repo_id": "",
        "repo_name": "",
        "repo_path": "",        # 本地路径 / 或挂载路径
        "branch": "main"
    },
    # ======================
    # 2️⃣ 需求分析产物
    # ======================
    "requirement_structured": {
        "goal": "",
        "features": [],
        "constraints": [],
        "acceptance_criteria": []
    },

    # ======================
    # 3️⃣ 代码库上下文（关键新增）
    # ======================
    "codebase_context": {
        "query": "",              # 本次检索/关注点
        "files": [
            {
                "path": "",
                "content": ""    # 裁剪后的代码片段
            }
        ]
    },

    # ======================
    # 4️⃣ 方案设计
    # ======================
    "design": {
        "architecture": "",
        "modules": [],
        "api_design": [],
        "file_change_plan": [],
        "risk_analysis": ""
    },

    # ======================
    # 5️⃣ 代码生成
```

```
    # ======================
    "code_diff": {
        "files_changed": [],
        "diff": ""
    },

    # ======================
    # 6️⃣ 测试
    # ======================
    "tests": {
        "unit_tests": "",
        "integration_tests": "",
        "test_results": {
            "passed": False,
            "logs": ""
        }
    },

    # ======================
    # 7️⃣ 评审
    # ======================
    "review": {
        "issues": [],
        "suggestions": [],
        "risk_level": "low",
        "pass": False
    },

    # ======================
    # 8️⃣ 交付
    # ======================
    "delivery": {
        "final_diff": "",
        "summary": "",
        "mr_url": ""
    }
}
```